

Abstract

Communication is gaining importance and influencing people all around the world, it's important to assure its safety.

There are currently a wide variety of applications that support a secure communication, however most require a third party participation.

Our solution focused on only using the elements necessary for a secure communication.

1 Introduction i

Nowadays communication has gained a fundamental weight in the various sociocultural, economic and political areas. The high criticality of communication impulsed the appearance of various solutions mainly based on a client-server architecture that requires the participation of third parties. This participation, in addition to having additional costs, puts at risk the security of the information. There are also some peer-to-peer solutions such as Skype, pTox that allow communication without third parties, but do not guarantee anonymity. Our solution wants to solve both problems. It's based on a Peer-to-Peer (P2P) architecture and is intended to remove third party participation, in a way that reduces costs, increases speed and mitigate some of the potential information security risks. Communication is carried out only at the network level, in order to mitigate the anonymity problem.

2 Fundamental Concepts

In this section "fundamental concepts" its aboarded all the concepts needed to understand the options adopted in developing the solution.

2.1 Architecture

Architecture is one of the fundamental aspects in the development of any software. In distributed systems the architecture is based in the premise of roles and well defined responsibilities, as follows:

2.1.1 Client-Server

In the Client Server, roles and responsibilities are well defined, illustrated in **Figure 1**.

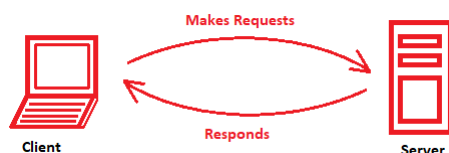


Figure 1: Architecture *Client-Server*

In this architecture, the client corresponds to the sender of the message and the server is responsible for delivering to the appropriate recipient(s),

2.1.2 Peer-to-Peer

The *Peer-to-Peer* architecture is based on the premise that all the members of the network, denominated by nodes, are the same[2], illustrated in **Figure 2**.

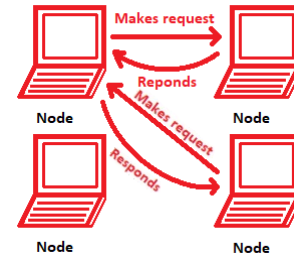


Figure 2: Peer-to-Peer Architecture

There are currently a large set of variations of *Peer-to-Peer*, namely:

- *Structured P2P*, nodes find each other through an hashing table.
- *Hybrid P2P*, some nodes have more responsibilities than others.
- *Pure P2P*, ll nodes have the same level of responsibility.
- *Centralized P2P*, a central entity is required in order to manage-ment the nodes on the network;

Between all the options we chose Pure P2P due to its simplicity and the great advantages it provides. In Pure P2P the communication is made from the sending node directly to the intended node.

2.2 Network

The purpose of *network* is to enable the communication of information in distributed systems. A network can be established regardless of the transmission medium (cable, fiber), infrastructure (routers, switch) or as-associated software. [2] *networks* can be characterized according to is dimension, as follows:

- *Personal area networks (PANs)*
- *Local area networks (LANs)*
- *Wide area networks (WANs)*
- *Metropolitan area networks (MANs)*

Our solution is based on a *LAN* which has the advantage of making direct connections between the elements of the same network segment, thus preventing message routing or redirecting throught the Internet.

2.3 Routing

Routing is the process of selecting a network path for traffic redirection. This process is based on the following delivery methods:

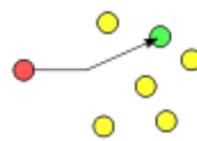


Figure 3: Unicast



Figure 4: Multicast

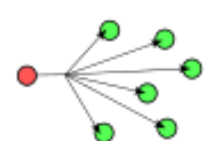


Figure 5: Broadcast

In order to process the introduction of a node in the network, it was necessary to develop a protocol that allowed the identification of existing nodes in the network. For this, the delivery methods *Broadcast* and *Unicast* applied on the layer *Data Link Access* were used.

2.4 Cryptography

Cryptography is the practice and study of techniques for secure communication in the presence of third parties. (Ronald L, 1990). [1]

2.4.1 Symmetric Cryptography

Symmetric Cryptography is based on the use of the same cryptographic key for message encryption and decryption.

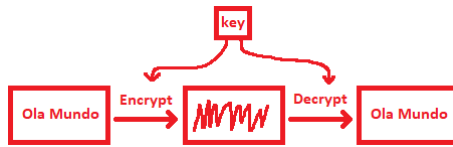


Figure 6: Symmetric Cryptography

This type of encryption is computationally light, fast and causes little latency on the network.

2.4.2 Asymmetric Cryptography

Asymmetric encryption, also known as public key cryptography, uses a complementary key pair. Both keys allow the processes of encryption and decryption of messages, however only with the other key its possible to reverse the operation performed. Generally the private key is kept secret while the public key is usually distributed.

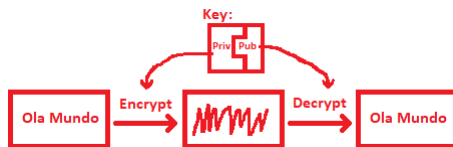


Figure 7: Asymmetric Cryptography

This cryptographic method is extremely computationally heavy and limited, but has the ability to create a secure channel that is difficult to attack.

3 Proof of Concept

In this section, all the processes inherent to the idealization and implementation of the developed solution will be explored.

3.1 Data structure

In a Peer to Peer network there the needs to exist an identification mechanism of peers. For this, the class **Contact** was developed, which can easily be distributed by the network, being composed of the following properties:

- *ID*, identification number;
- *PC-NAME*, name of the computer in the network;
- *MAC*, network card identification;
- *IP*, last connected ip address;
- *Public-key*, public asymmetric key that allows the creation of a secure channel.

The information contained in a **Contact** only allows it to be identified it on the network, so a data structure capable of storing all known contacts and ensuring their persistence is required. For this, based on the ssh protocol the following files were created:

- *rsa/private*, stores only the private key(s);
- *rsa/public*, stores known contacts, including your own;

Additionally, given that not all Contacts are available all the time, there is a in memory data structure **Hashmap** that maps the active nodes of the network and list them so the user can create a connection,

3.2 Introduction on the Network

One of the major problems with the architecture *Pure Peer to Peer* is the process of introducing a node into the network and the ability of the network to adapt to it. For this, it was necessary to develop a protocol that will be explained below.

3.2.1 Introduction

In the introduction process, the new node must demonstrate its intention to join the network, in order to do that, it sends its *Contact* in *Broadcast* via *UDP* protocol .

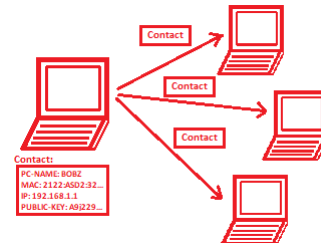


Figure 8: Introduction of Contact

3.2.2 Interpretation

In the interpretation step, each active node in the network processes the new contact, applying the following changes:

- *rsa/public*, add/change corresponding entry;
- *hashmap*, maps new node as active in the network;

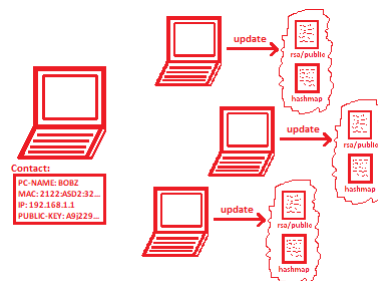


Figure 9: Processing New Contact

3.2.3 Integration

In the integration process each node after processing the new contact, sends through the *TCP* protocol a *Unicast* with its own contact, in order to establish a possible future connection.

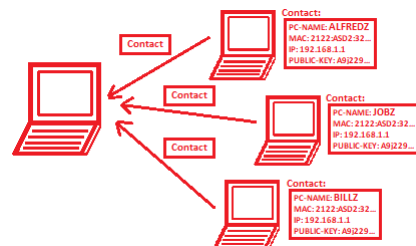


Figure 10: Integration of new Contact

3.3 Communication

After identifying the active nodes in the network, the application allows the creation of communication channels.

3.3.1 Creation of a Secure Channel

In order to initialize a communication it is necessary to generate a secure channel. For this, we used the receiver's public key and the emitter's private key, shown in **Figure 11**.

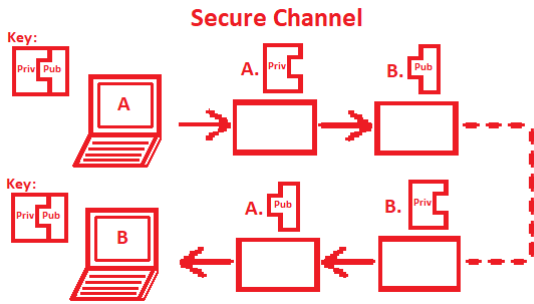


Figure 11: Creation of a Secure Channel

The receiver in order to obtain the information sent in the secure channel, must reverse the process sender used, that is, use the private key and then the public key of the sender. This channel is secure, however it is based on asymmetric cryptography which is not ideal.

3.3.2 Key Exchange

Using the previously generated secure channel, the emitter node generates a symmetric key and sends it. This symmetric key will be used from now on, until the communication is finished.

3.4 Tests

In this chapter some of the tests carried out to ensure the normal functioning of the application will be addressed.

3.4.1 Node Recognition

To carry out this test, the computer "Desktop-QUACKMAN" started the application. It applied the Introduction process, but since there was no node on the network, no contacts were added. This process is shown in **Figure 12**.

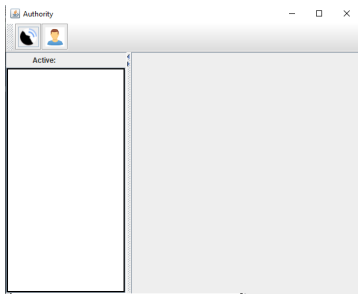


Figure 12: Contacts of DESKTOP-QUACKMAN

Then the computer "Desktop-Marquez" opens the application and both computers update their contacts as expected.

Note: During this test, we verified that firewalls can block the process of identifying elements on the network.

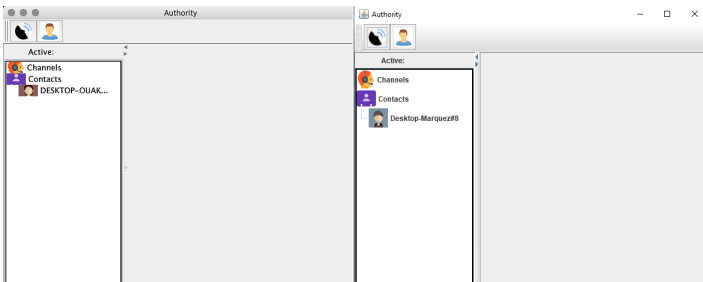


Figure 13: Contacts of both computers

3.4.2 Creation of a Communication

DESKTOP-QUACKMAN selects the contact joaos-mbp.lan. The application creates a corresponding channel in "Channel", shown in **Figure 14**.

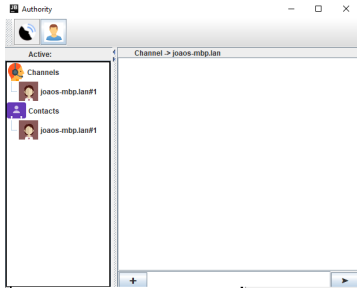


Figure 14: Creation of a Channel

3.4.3 Chat Conversation

The user DESKTOP-QUACKMAN, after creating a channel, exchanges messages with the user joaos-mbp.lan.

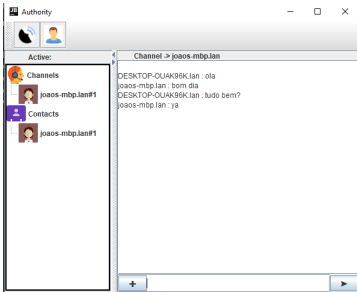


Figure 15: Example of chat usage

3.5 Discussion of Results

Initially, the use of certificates for identifying nodes on the network was considered. However, due to their creation, sharing and storage, it became an unfeasible option. The non-use of certificates prevented the use of the secure communication protocol TLS/SSH, so it was necessary to create the entire infrastructure that would allow its creation.

To circumvent the problem, Java's Crypto library was used, which allows the creation of various types of symmetric and/or asymmetric keys. Due to the way the RSA algorithm works, it prevented the successive application of the encryption process, thus making it difficult to create the secure channel.

To get around this situation, it was necessary to split the bytes resulting from the first encryption into two parts and apply a second encryption to each.

We had to apply this process because of the way the RSA encryption algorithm itself works, which, being a *Block Cipher*, the encryption result always produces a set of bytes with a minimum size.

Unfortunately, it was not possible to complete the file upload and download process via chat.

3.6 Conclusion

With the elaboration of this work it was possible to learn about distributed architectures and in detail the variations of Peer to Peer.

Additionally, it was possible to gain experience with the use of cryptographic algorithms, namely RSA and AES.

We suggest continuing this type of work since it allows us to explore in some detail certain technologies and areas taught in the Curricular Unit.

References

[1] Ronald L. Rivest. *Cryptography*. Elsevier, 1990. ISBN 0849385237.
In: J. Van Leeuwen. *Handbook of Theoretical Computer Science*.
[2] Steen Tanenbaum, A. S. *Distributed Systems: Concepts and Design*. Addison-Wesley, 2005. ISBN 0849385237.